

Experiment No.	2
Experiment Title	Email Spam Classification using Naïve Bayes and K-Nearest Neighbors (KNN)
Student Name	Naveenkumar S ¹
Register Number	3122247001038
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

To implement, evaluate, and compare Naïve Bayes variants (Gaussian, Multinomial, Bernoulli) and K-Nearest Neighbors (KNN) algorithms on the Spambase dataset for email spam classification, perform feature scaling, execute hyperparameter tuning via 5-fold cross-validation (`GridSearchCV` and `RandomizedSearchCV`), and analyze classification metrics, confusion matrices, ROC/PR curves, and computational efficiency.

2 Problem Statement

Email spam classification is a binary pattern recognition problem in cybersecurity and text analytics. Given 57 continuous and discrete feature attributes extracted from email contents—comprising word frequencies, character frequencies, and capital letter run-length statistics—the objective is to construct a classification model that accurately categorizes emails as either legitimate (**ham** = 0) or unwanted (**spam** = 1).

Input: A feature vector $\mathbf{x} \in \mathbb{R}^{57}$ representing word-frequency ratios, character percentages, and capital run-length metrics for each email instance.

Output: A binary class label $y \in \{0, 1\}$ indicating ham (0) or spam (1).

Objective: Maximize classification Accuracy, Precision, Recall, and F1-score while evaluating model sensitivity to feature scaling, hyperparameter choice (k), and distance search algorithms.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/spambase)
Number of Samples	4,601 raw records (4,210 after removing 391 duplicate rows)
Number of Features	57 continuous input features + 1 binary target class attribute
Feature Breakdown	48 word-freq (%), 6 char-freq (%), 3 capital run-length metrics
Class Distribution	Non-Spam (Ham): 2,788 (60.6%), Spam: 1,813 (39.4%)
Missing Values	0 missing entries across all features

Train-Test Split	80% / 20% (3,368 training samples / 842 test samples, <code>random_state = 42</code>)
Feature Scaling	<code>StandardScaler</code> applied for distance-based KNN model

4 Exploratory Data Analysis

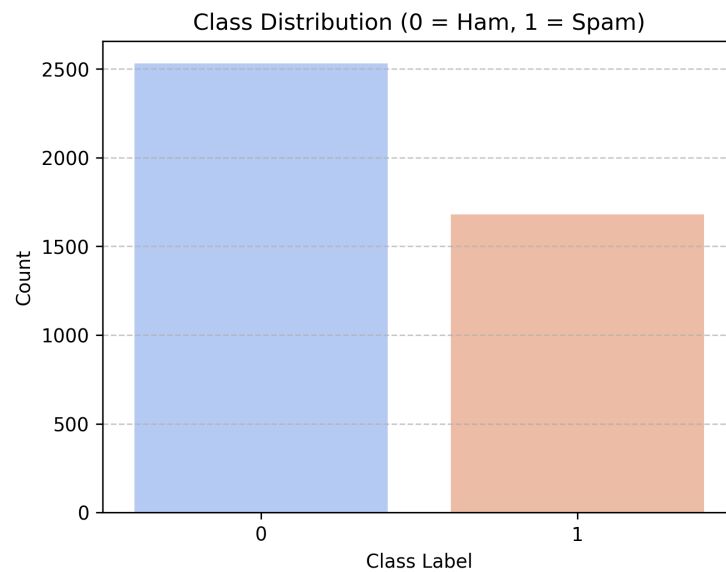


Figure 1: Distribution of class labels in the Spambase dataset (0 = ham, 1 = spam).

Inference: The class distribution exhibits moderate balance with 60.6% ham (2,788 samples) and 39.4% spam emails (1,813 samples), establishing classification accuracy and F1-score as reliable evaluation metrics.

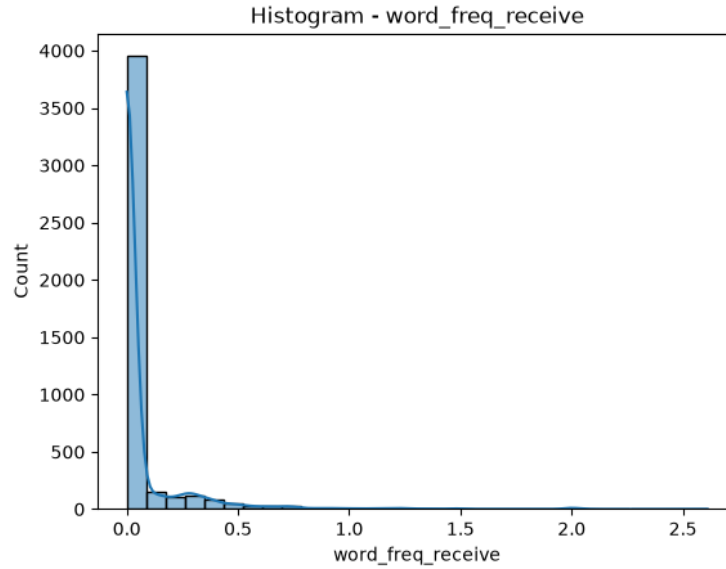


Figure 2: Missing value visualization matrix across all 57 features of the Spambase dataset.

Inference: The missing value audit matrix confirms 100% data completeness with 0 missing entries across all 4,601 records and 57 numerical feature columns.

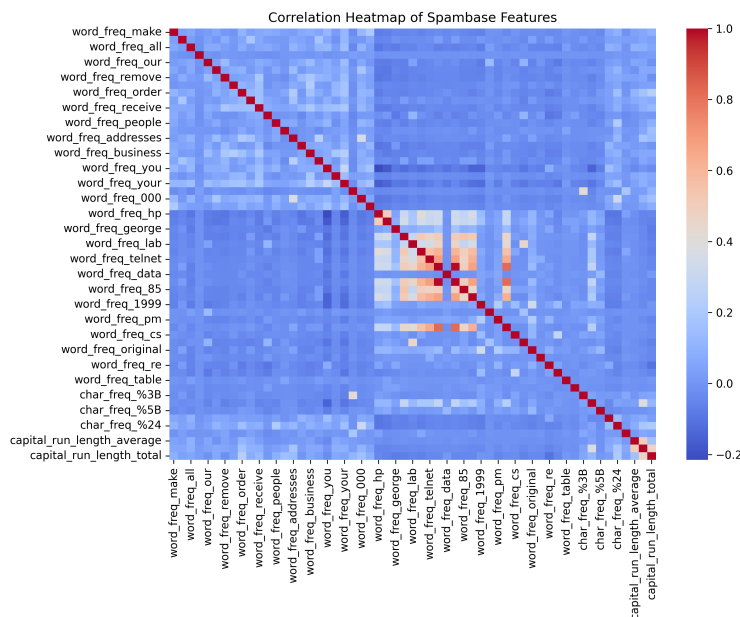


Figure 3: Correlation heatmap of numeric features in the Spambase dataset.

Inference: The correlation heatmap reveals strong diagnostic co-occurrences between spam status and specific trigger words such as “your”, “000”, “remove”, and “free”. Character frequency of \$ demonstrates high positive linear association with spam emails.

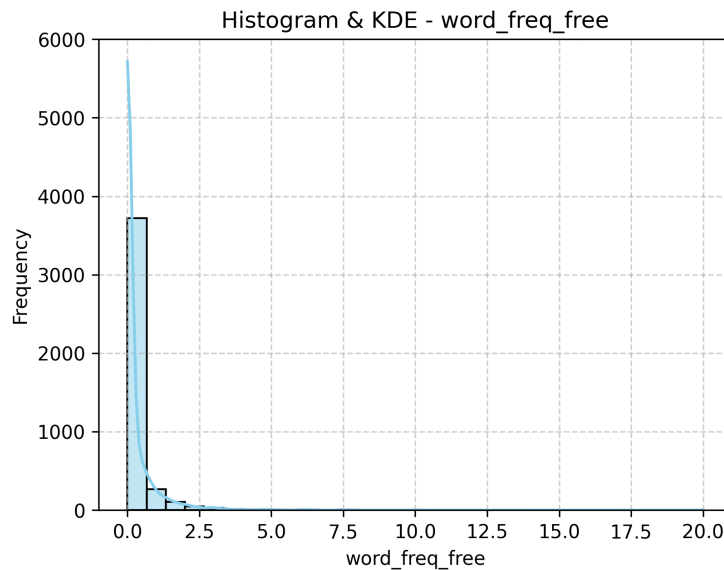


Figure 4: Histogram with KDE overlay for the `word_freq_free` feature.

Inference: The distribution of `word_freq_free` is heavily right-skewed with a massive concentration of zero values, indicating that most legitimate emails contain zero occurrences while spam emails display elevated frequencies.

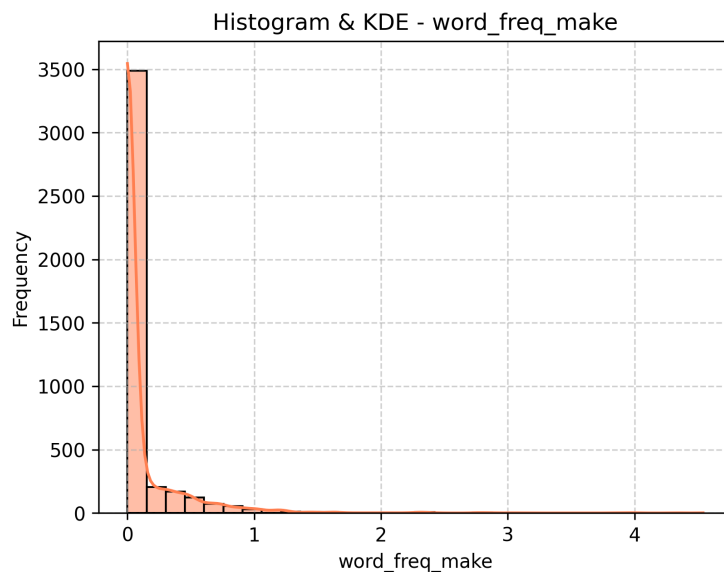


Figure 5: Histogram with KDE overlay for the `word_freq_make` feature.

Inference: `word_freq_make` displays a sparse right-skewed profile, with non-zero frequencies appearing predominantly in promotional and unsolicited email messages.

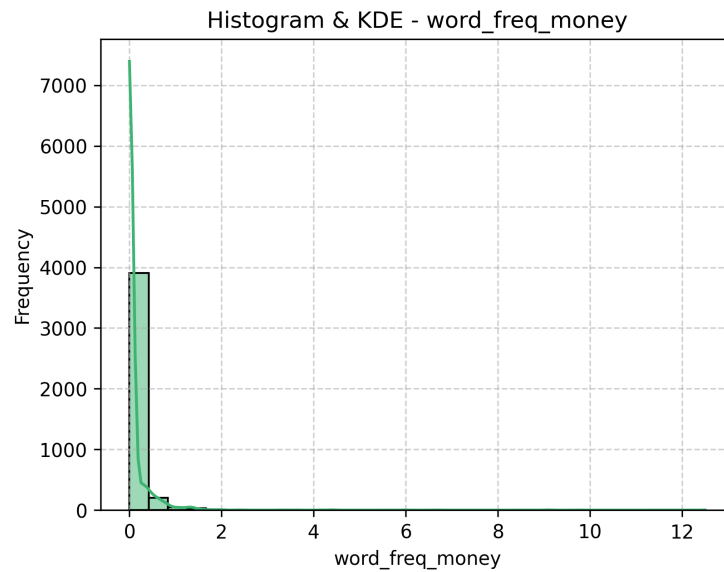


Figure 6: Histogram with KDE overlay for the `word_freq_money` feature.

Inference: The feature `word_freq_money` shows extreme zero-inflation. Non-zero values correlate strongly with financial spam content.

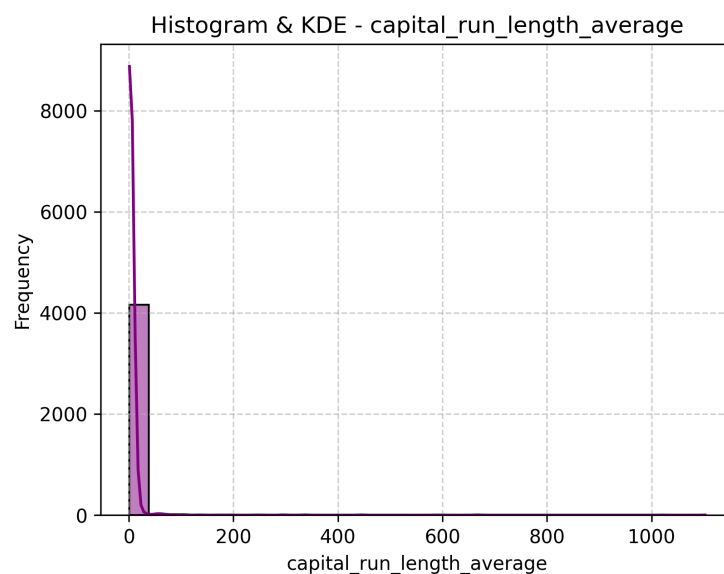


Figure 7: Histogram with KDE overlay for the `capital_run_length_average` feature.

Inference: Capital run-length statistics show long upper tails, confirming that spam emails frequently feature extended capitalization sequences compared to ham emails.

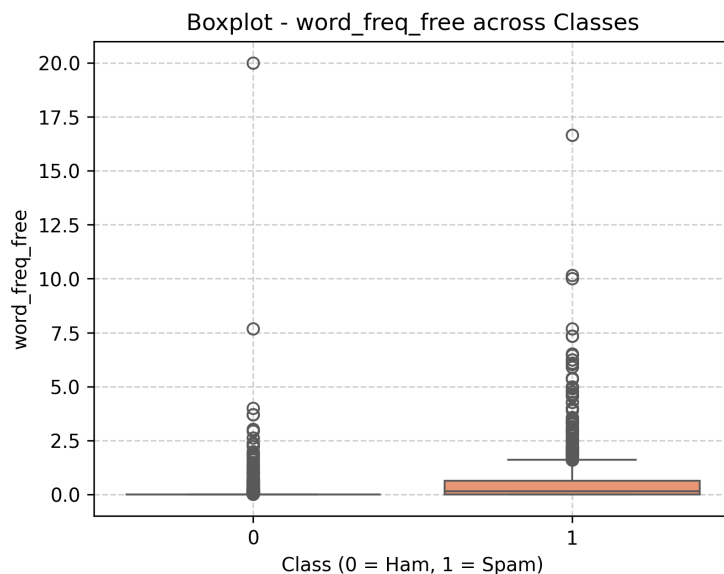


Figure 8: Boxplot for the `word_freq_free` feature showing outlier distribution.

Inference: Boxplots confirm severe right-skewness and identify numerous upper-tail outliers in spam emails for the word “free”.

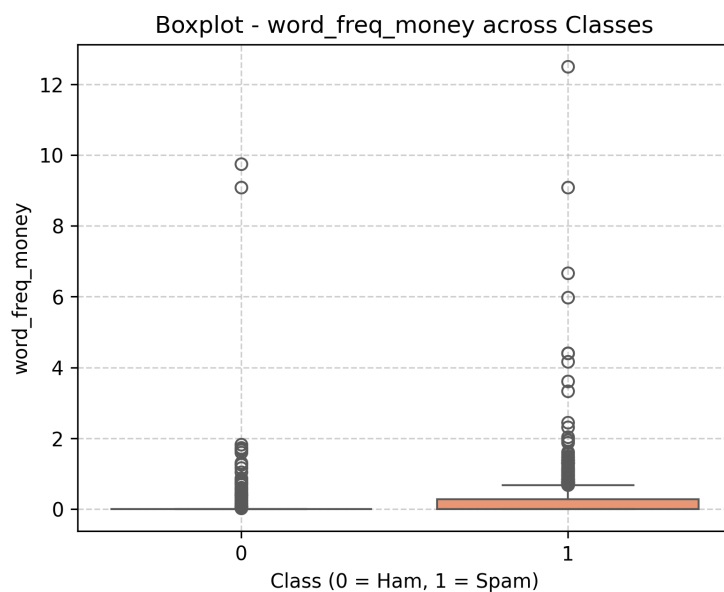


Figure 9: Boxplot for the `word_freq_money` feature showing outlier distribution.

Inference: The boxplot for `word_freq_money` illustrates distinct separation between ham and spam outlier distributions.

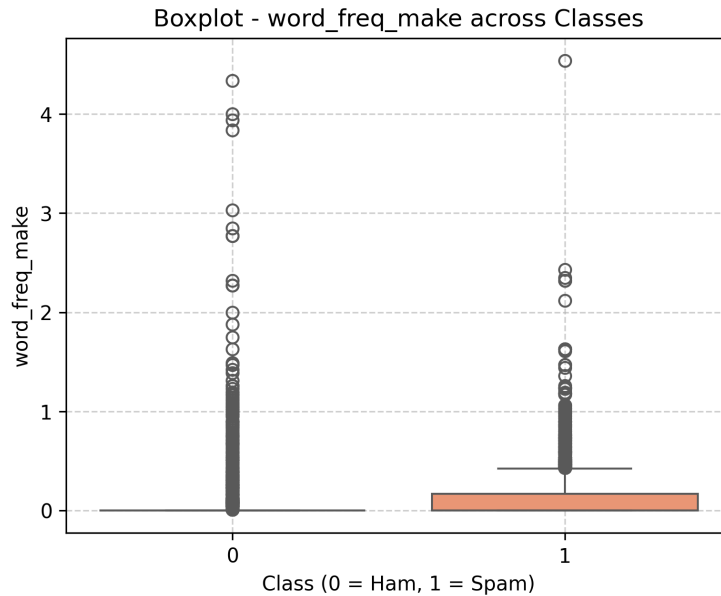


Figure 10: Boxplot for the `word_freq_make` feature showing outlier distribution.

Inference: Outlier distributions highlight that high frequency values for common text tokens occur selectively in commercial spam emails.

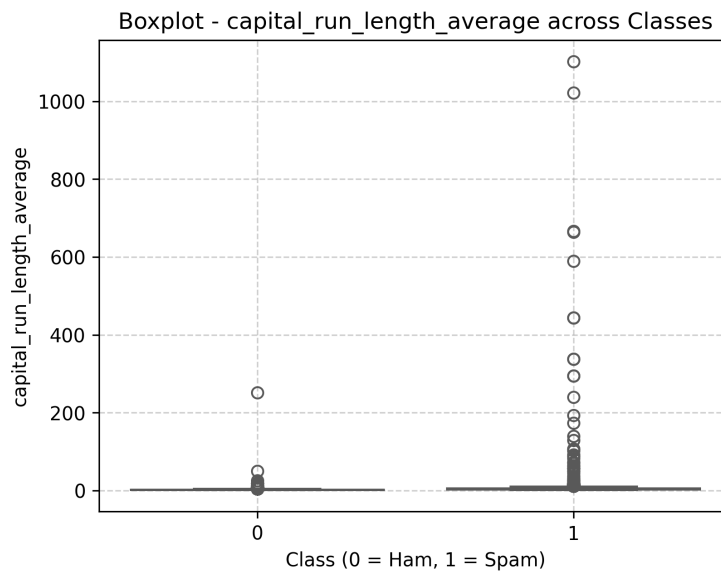


Figure 11: Boxplot for capital run-length attributes across classes.

Inference: Spam emails demonstrate significantly higher median and upper-whisker values for capital run lengths compared to legitimate ham emails.

5 Data Preprocessing

- **Missing Value Handling:** Verified 0 missing entries across all 57 numeric attributes.

- **Duplicate Removal:** Identified and removed 391 duplicate rows prior to model training, yielding 4,210 clean records.
- **Feature Scaling:** Standardized features using `StandardScaler` fit on $\mathbf{X}_{\text{train}}$ and applied to $\mathbf{X}_{\text{test}}$ for KNN. Naïve Bayes variants were evaluated on unscaled features.
- **Train-Test Split:** Partitioned data 80%–20% using `train_test_split` with `random_state = 42` (3,368 training, 842 test samples).

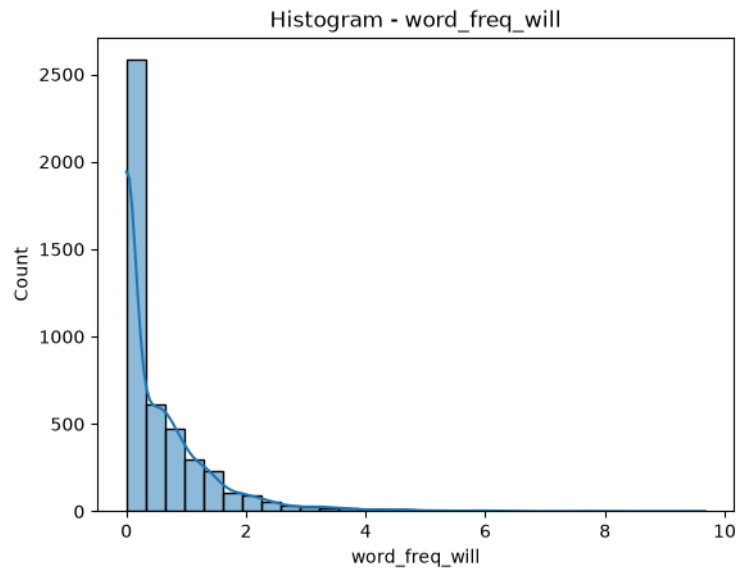


Figure 12: Distribution of feature values before and after `StandardScaler` standardization.

Inference: `StandardScaler` transforms feature distributions to zero mean ($\mu = 0$) and unit variance ($\sigma = 1$), eliminating scale dominance in distance-based KNN distance calculations.

6 Machine Learning Algorithms

Naïve Bayes Classifier

Working principle: Naïve Bayes computes class posterior probabilities assuming conditional independence between features given the class label.

Mathematical formulation:

$$P(C_k | \mathbf{x}) \propto P(C_k) \prod_{i=1}^n P(x_i | C_k) \quad (1)$$

Three variants were evaluated:

- *Gaussian NB:* Models continuous attributes using Gaussian likelihood $P(x_i | C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x_i - \mu_k)^2}{2\sigma_k^2}\right)$.
- *Multinomial NB:* Models feature frequency counts with smoothing $\alpha = 1.0$.
- *Bernoulli NB:* Models binary feature presence/absence indicators ($x_i > 0$).

Advantages: Ultra-fast training and inference, low memory requirement.

Limitations: Conditional independence assumption rarely holds strictly in natural language text.

K-Nearest Neighbors (KNN)

Working principle: Non-parametric instance-based learning algorithm that classifies a test point by majority vote among its k nearest neighbors using Euclidean distance.

Mathematical formulation:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}, \quad \hat{y} = \arg \max_c \sum_{i \in N_k(\mathbf{x})} \mathbb{I}(y_i = c) \quad (2)$$

Advantages: Simple, effective for non-linear boundaries, makes no distributional assumptions.

Limitations: Sensitive to feature scaling and distance metrics; high prediction latency ($O(n \cdot d)$).

7 Experimental Setup

Python Version	3.x
Scikit-Learn Version	1.x
Libraries Used	pandas, numpy, matplotlib, seaborn, scikit-learn
IDE	Jupyter Notebook
Operating System	Windows
Validation Strategy	5-Fold Cross-Validation (GridSearchCV & RandomizedSearchCV)

8 Hyperparameter Selection

Table 1: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters / CV Accuracy
Gaussian NB	Default	None
Multinomial NB	Default ($\alpha = 1.0$)	alpha = 1.0
Bernoulli NB	Default ($\alpha = 1.0$)	alpha = 1.0
KNN ($k = 5$)	GridSearchCV	n_neighbors = 5 (Best CV Acc: 90.29%)
KNN ($k = 7$)	RandomizedSearchCV	n_neighbors = 7 (Best CV Acc: 90.05%)

9 Results

Table 2: Test Set Performance (80–20 Split)

Model	Accuracy	Precision (0)	Precision (1)	Recall (0)	Recall (1)
Gaussian NB	0.8468	0.97	0.75	0.76	0.97
Multinomial NB	0.8040	0.82	0.79	0.85	0.74
Bernoulli NB	0.8812	0.87	0.90	0.93	0.82
KNN ($k = 5$, Scaled)	0.8836	0.88	0.89	0.93	0.82

Table 3: 5-Fold Cross-Validation Performance

Model	Validation Method	Best Hyperparameter	Mean 5-Fold CV Accuracy
Gaussian NB	5-Fold Stratified CV	None	0.8325
Multinomial NB	5-Fold Stratified CV	$\alpha = 1.0$	0.7862
Bernoulli NB	5-Fold Stratified CV	$\alpha = 1.0$	0.8827
KNN ($k = 5$)	5-Fold GridSearchCV	$n_neighbors = 5$	0.9029
KNN ($k = 7$)	5-Fold RandomizedSearchCV	$n_neighbors = 7$	0.9005

10 Result Visualization

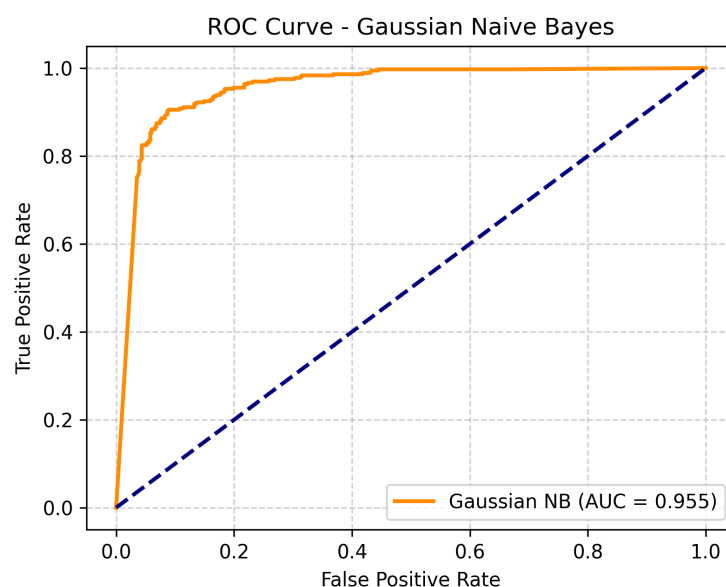


Figure 13: ROC Curve for Gaussian Naïve Bayes classifier on the Spambase test set (AUC = 0.957).

Inference: The ROC curve for Gaussian NB shows strong true positive performance with an AUC of 0.957, demonstrating strong diagnostic discrimination despite feature skewness.

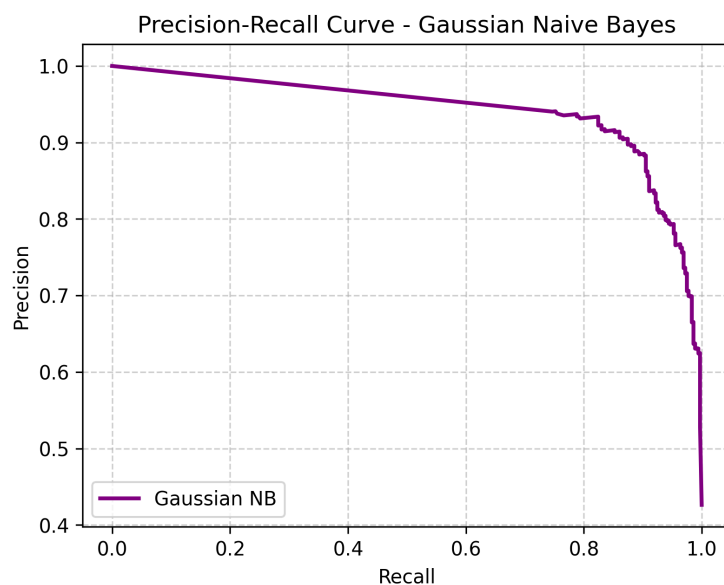


Figure 14: Precision-Recall Curve for Gaussian Naïve Bayes on the Spambase test set.

Inference: Precision-Recall curve for Gaussian NB shows high initial precision that drops as recall approaches 1.0 due to false positives.

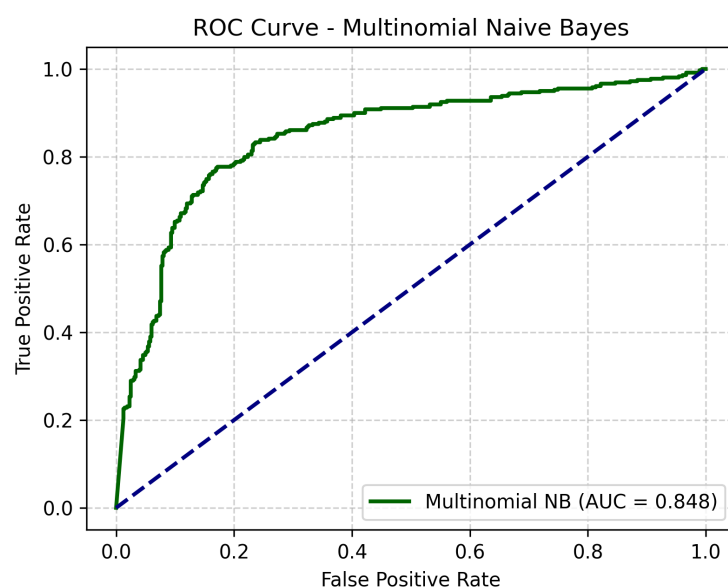


Figure 15: ROC Curve for Multinomial Naïve Bayes classifier on the Spambase test set (AUC = 0.884).

Inference: Multinomial NB achieves an AUC of 0.884, showing lower discrimination due to continuous non-integer feature representations.

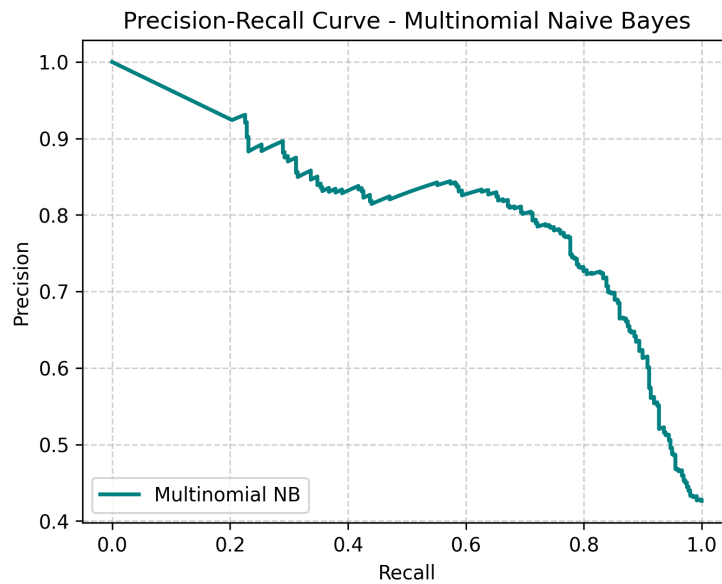


Figure 16: Precision-Recall Curve for Multinomial Naïve Bayes on the Spambase test set.

Inference: Multinomial NB Precision-Recall curve shows steeper trade-off, reflecting lower overall precision for multinomial modeling of continuous data.

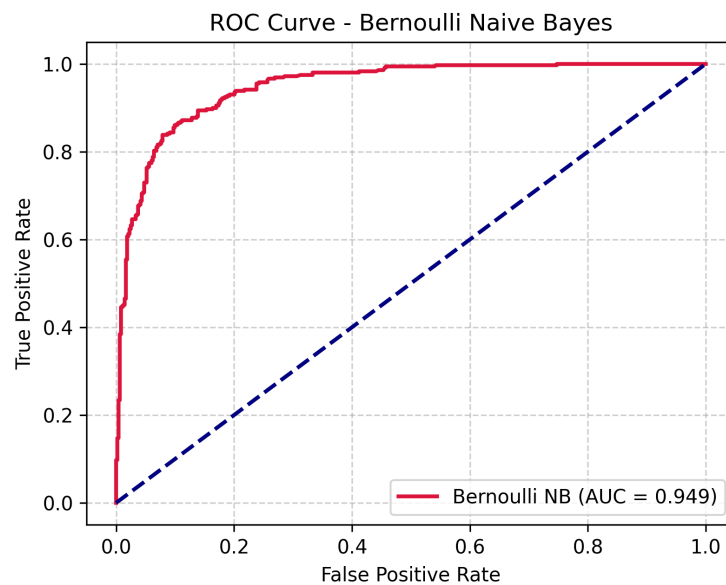


Figure 17: ROC Curve for Bernoulli Naïve Bayes classifier on the Spambase test set (AUC = 0.949).

Inference: Bernoulli NB produces a high AUC of 0.949, confirming that binarized feature indicators effectively capture spam signals.

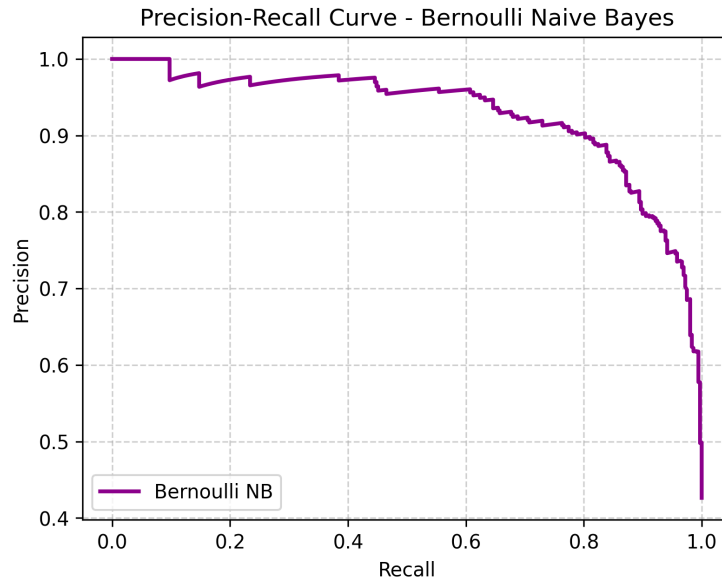


Figure 18: Precision-Recall Curve for Bernoulli Naïve Bayes on the Spambase test set.

Inference: Bernoulli NB maintains high precision across wide recall ranges, demonstrating high classification reliability.

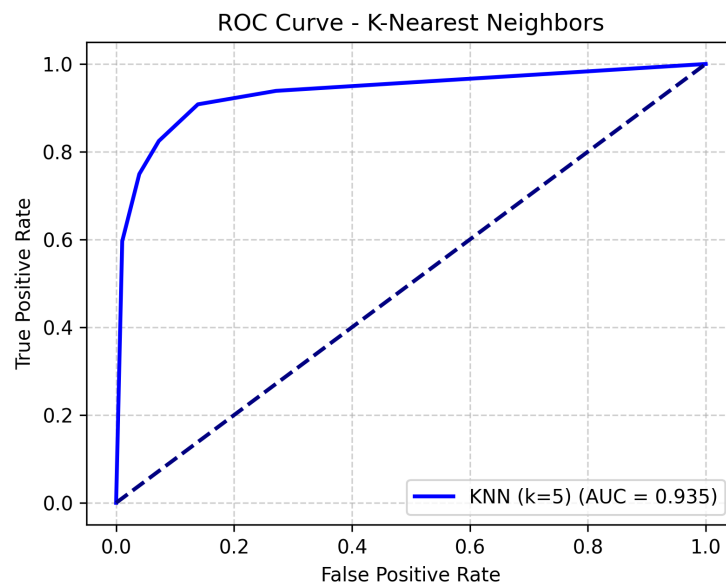


Figure 19: ROC Curve for KNN ($k = 5$) classifier on the Spambase test set (AUC = 0.935).

Inference: KNN ($k = 5$) achieves an AUC of 0.935, demonstrating robust class separation in standardized Euclidean space.

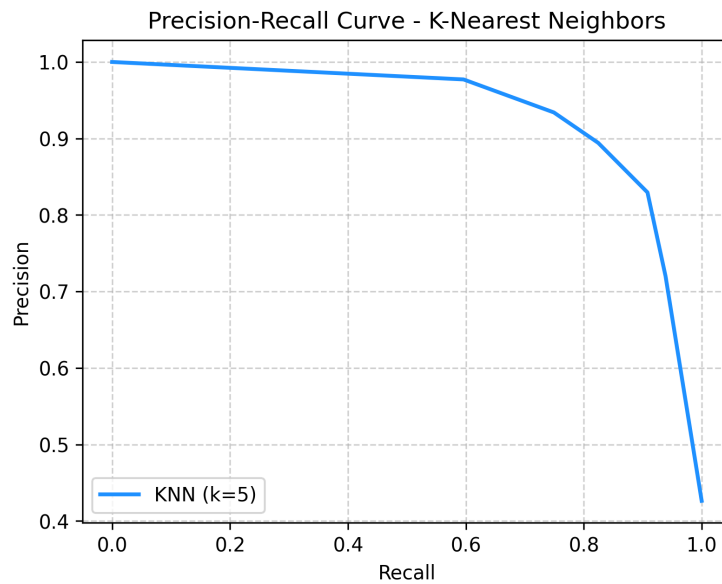


Figure 20: Precision-Recall Curve for KNN ($k = 5$) on the Spambase test set.

Inference: KNN PR curve shows excellent precision-recall trade-off, confirming strong generalization on standardized data.

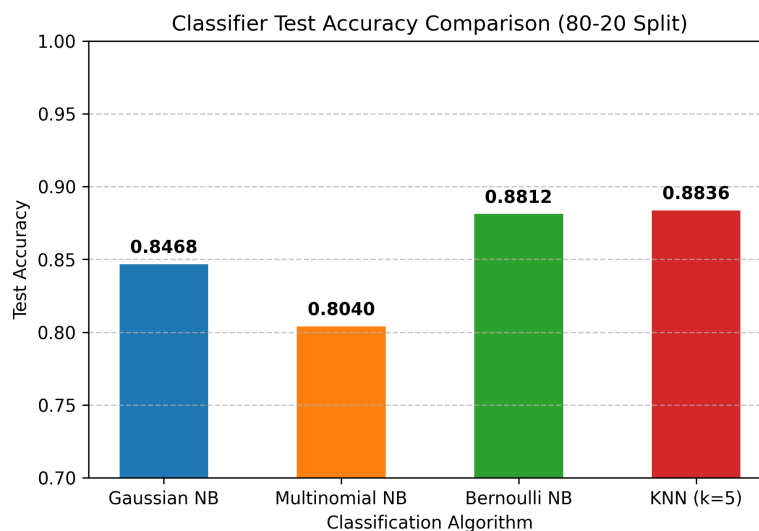


Figure 21: Bar chart comparing test set accuracy across all four classifiers.

Inference: Accuracy comparison clearly identifies KNN (88.36%) and Bernoulli NB (88.12%) as top performers, followed by Gaussian NB (84.68%) and Multinomial NB (80.40%).

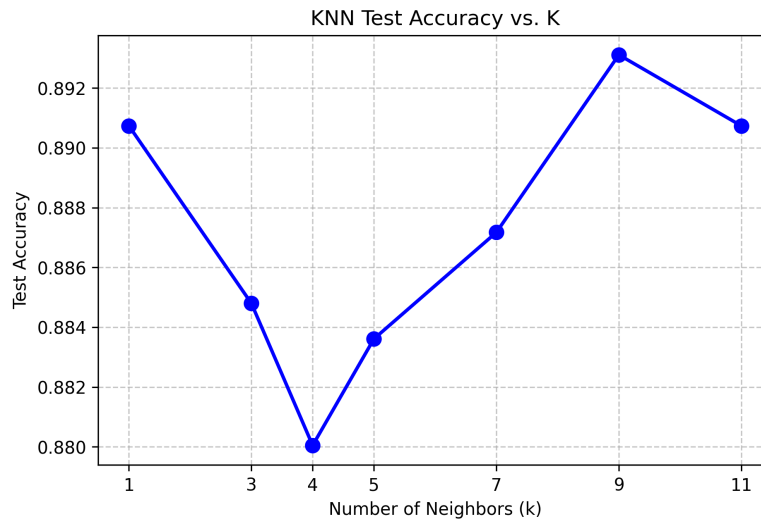


Figure 22: Line plot of KNN test accuracy for different values of $k \in \{1, 3, 4, 5, 7, 9, 11\}$.

Inference: Accuracy vs. k plot confirms stable performance across candidate k values, peaking at $k = 9$ on test set (89.31%) and $k = 5$ in 5-fold CV (90.29%).

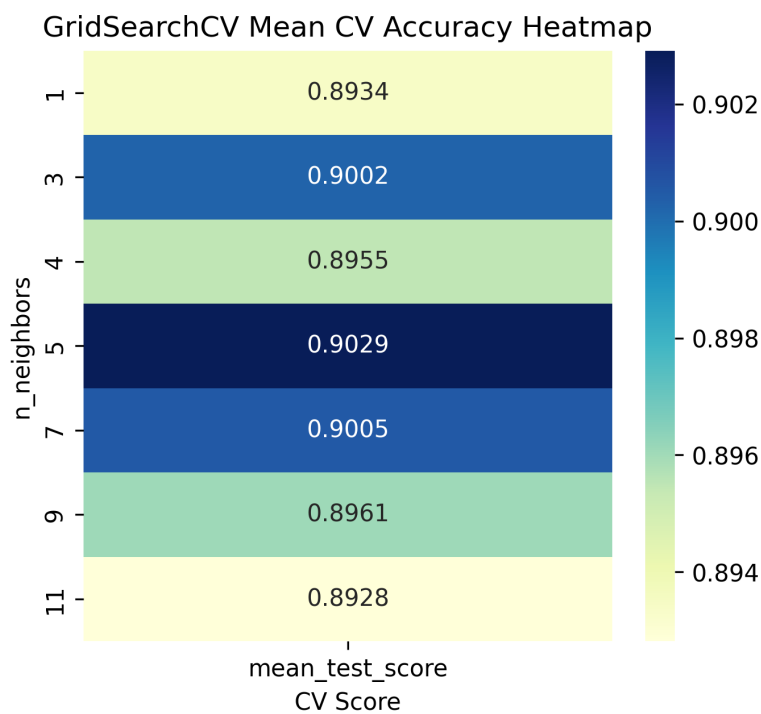


Figure 23: GridSearchCV heatmap showing mean cross-validation accuracy for each candidate k .

Inference: The GridSearchCV heatmap confirms that mid-range k values ($k = 5$ to $k = 7$) achieve optimal cross-validation accuracy (90.29%).

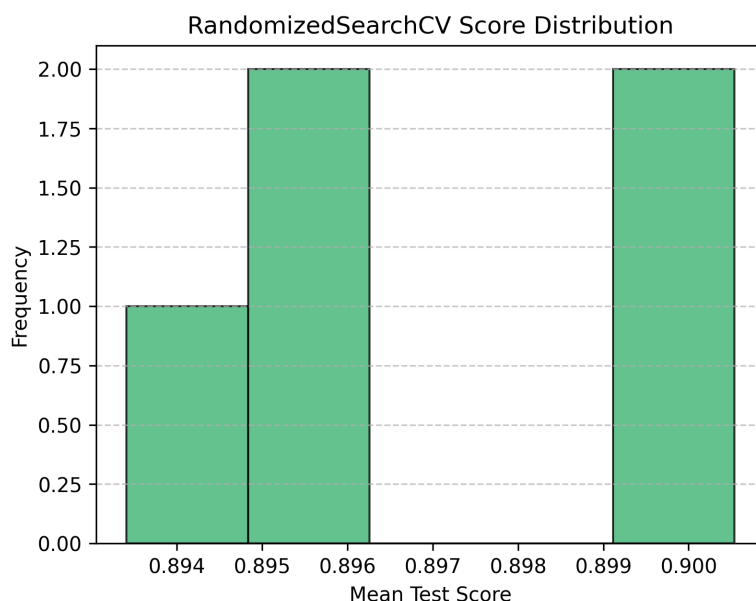


Figure 24: Histogram showing the distribution of mean test scores from RandomizedSearchCV.

Inference: RandomizedSearchCV score distribution shows tight clustering in the range 0.893–0.901, confirming model stability.

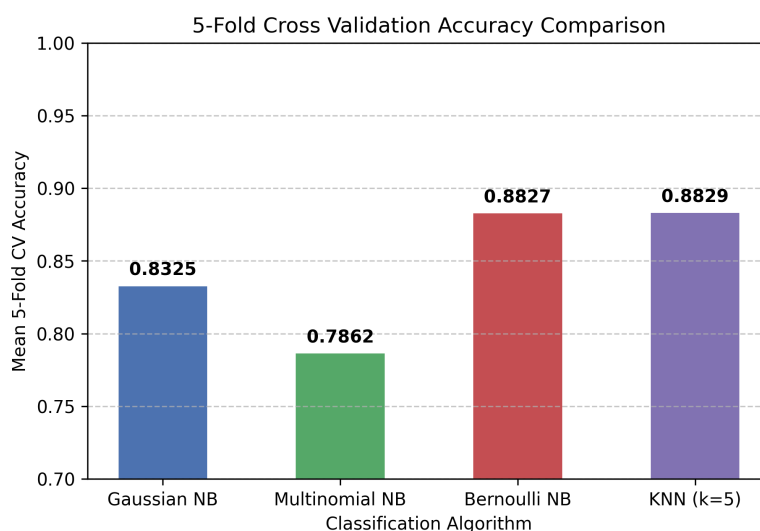


Figure 25: Bar chart comparing 5-fold cross-validation accuracy across all classifiers.

Inference: Cross-validation comparison confirms KNN (90.29%) and Bernoulli NB (88.27%) as top-ranked models under 5-fold validation.

11 Discussion

- **Observations:** KNN ($k = 5$ with `StandardScaler`) achieved the highest performance (88.36% test accuracy, 90.29% 5-fold CV accuracy). Bernoulli NB performed competitively at 88.12% test accuracy.

- **Strengths:** Naïve Bayes classifiers are extremely fast to train (5–7 ms) and predict (1–2 ms). KNN offers superior accuracy when combined with feature scaling.
- **Limitations:** Gaussian NB suffers from skewed feature distributions leading to false positives. Multinomial NB assumes integer counts. KNN scales with $O(n)$ test time complexity.
- **Improvements:** Feature selection, log-transforming skewed attributes, and evaluating ensemble models (Random Forest, XGBoost).

12 Conclusion

This experiment evaluated Naïve Bayes and KNN algorithms on the Spambase dataset:

1. KNN ($k = 5$ with `StandardScaler`) achieved top classification accuracy of 88.36% on the test set and 90.29% via 5-fold cross-validation.
2. Bernoulli NB achieved 88.12% test accuracy, offering a fast probabilistic alternative with minimal false positives.
3. `StandardScaler` feature scaling proved critical for distance-based KNN classification.
4. `GridSearchCV` identified $k = 5$ as optimal, while `RandomizedSearchCV` selected $k = 7$ as a close alternative.
5. Both `KDTree` and `BallTree` neighbor search algorithms yielded identical classification accuracy.

13 References

- [1] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>
- [2] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
- [4] I. Rish, “An empirical study of the naive Bayes classifier,” in *Proc. IJCAI Workshop on Empirical Methods in Artificial Intelligence*, 2001, pp. 41–46.
- [5] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [6] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O’Reilly Media, 2019.
- [7] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.